Q14. K-means + KNN as Semi-Supervised Learning for Cultural Tourism

> **Question.** Can K-means clustering and KNN be combined into semi-supervised learning? Can we use it to cluster tourist destinations and organise a tour from a cultural point of view? How can Euclidean and Hamming distance be combined here? How can we introduce systemic creativity? Refer to Diagram 6 and modify it into a distribution based on cultural preferences.

1. The idea in plain words

We have many destinations but only a few carry a cultural label. **K-means** needs no labels: it simply finds natural groups. A handful of **labelled seed cities** tells us what each group means. **KNN** then checks every city against its nearest neighbours, fixes doubtful labels and labels brand-new cities. Structure comes from all the data and meaning comes from a few labels. That mix is exactly **semi-supervised learning**.

2. Assumptions

- The 20 cities of Diagram 6. Growth (%), spend per visit ($) and visits (millions, from the bubble size) are **approximate readings** of the chart.
- Each city gets 8 yes/no cultural attributes (our judgement): heritage, religious, art, street food, shopping, nightlife, nature, skyline.
- Only **4 of 20** cities are labelled, one per category: Rome = Heritage & Faith, Paris = Art & Lifestyle, Bangkok = Street Food & Markets, Dubai = Modern & Shopping.
- The traveller is a "culture lover" (heritage, religious, art, nature) who spends at most $1,200 per visit.

3. Design: who does what

Component	Job in this system
K-means (k = 4)	Groups all 20 cities without any labels. Centre = mean for numbers, majority vote for yes/no attributes.
4 seed labels	Name each cluster: the seed closest to a cluster centre gives it its name.
KNN (k = 3)	Re-checks each city: if 2 of its 3 nearest cities agree on a different label, the label moves. Also labels new cities.
Euclidean distance	Compares the numeric part: growth, spend and visits (min-max normalised).
Hamming distance	Compares the cultural part: how many of the 8 yes/no attributes differ.
Tour planner	Picks the cluster that best fits the traveller, then orders the stops by nearest neighbour.

4. Combining Euclidean and Hamming distance

Numbers and yes/no facts cannot share one ruler, so we measure each with its own ruler and blend them:

$$d(a, b) = \alpha \cdot \text{Euclid}(\text{numeric}_a, \text{numeric}_b) / \sqrt{3} + (1 - \alpha) \cdot \text{Hamming}(\text{bits}_a, \text{bits}_b) / 8$$

Both parts are scaled to 0–1. We use **$\alpha = 0.3$**, so culture carries 70% of the weight because this is a *cultural* tour. Real values from the run:

- **d(Rome, Prague) = 0.111**: they differ in only 1 of 8 attributes (Hamming 1/8 = 0.125) and their numbers are close.
- **d(Rome, Dubai) = 0.804**: 7 of 8 attributes differ (Hamming 0.875) and Dubai's spend is far higher.

5. Step-by-step approach

1. Normalise the numeric features and encode the 8 cultural bits for every city.
2. Run K-means (k = 4) on all 20 cities with the mixed distance d.
3. Name the clusters with the 4 seed cities, which gives every city a pseudo-label.
4. KNN (k = 3) re-checks every pseudo-label and moves a city when a clear majority disagrees. Repeat until nothing moves.
5. Use KNN to label new destinations that were never in the data.
6. Score each city against the traveller, pick the best cluster, add one cross-cluster stop, and route by nearest neighbour.

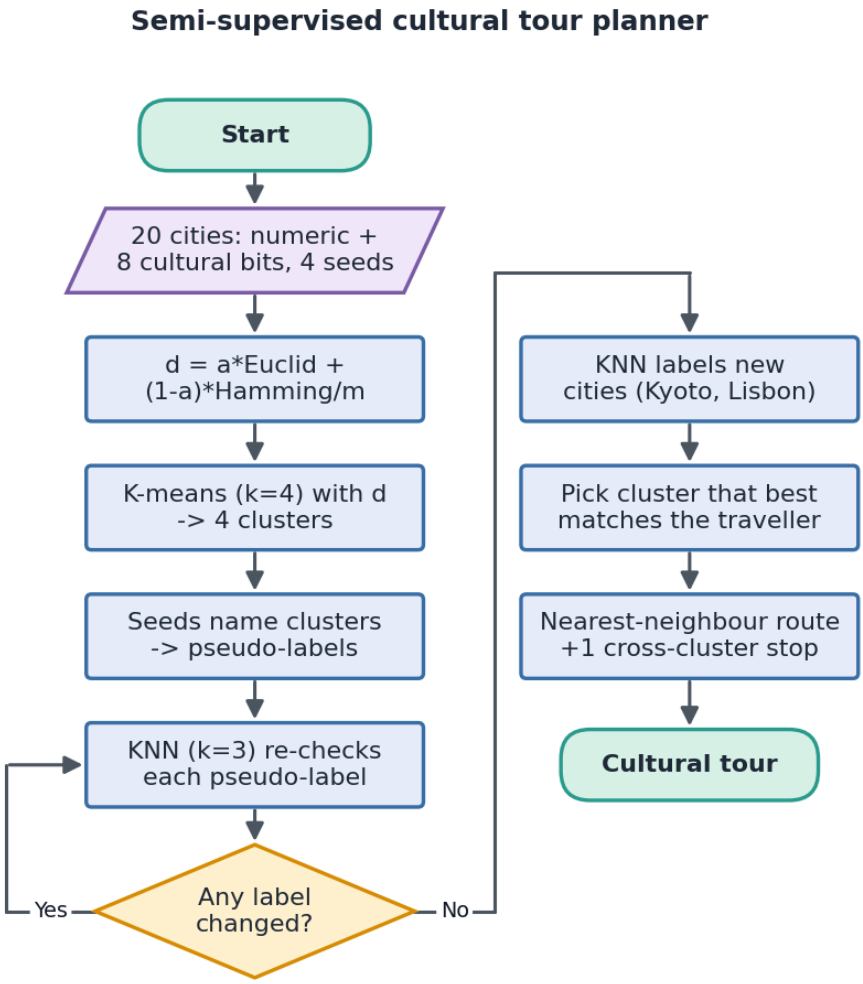


Figure 1. Flowchart of the semi-supervised cultural tour planner.

6. Pseudocode

```
function SemiSupervisedClusters(cities, seeds, k = 4, α = 0.3):
    d(a, b) = α·Euclid(num_a, num_b)/√3 + (1−α)·Hamming(bits_a, bits_b)/8
    clusters ← KMeans(cities, k, distance d)     # centre: mean of numbers, majority of bits
    name each cluster by the seed city closest to its centre
    label(c) ← name of c's cluster, for every city    # pseudo-labels
    repeat
        for each city c that is not a seed:
            N ← the 3 nearest cities to c under d       # KNN
            if at least 2 cities in N share a label L ≠ label(c): label(c) ← L
    until no label changes
    for each new city x: label(x) ← majority label of its 3 nearest cities
    return label

function PlanTour(profile, label, budget):
    match(c) = 1 − Hamming(profile, bits_c) / 8
    C* ← cluster with the highest average match
    stops ← cities of C* within budget + best-matching city from another cluster
    route ← start at the best match, then always go to the nearest unvisited stop
    return route
```

7. Can K-means and KNN be combined into semi-supervised learning? Yes

Here is what the real run did with only 4 labels:

Cluster (named by its seed)	K-means centre (attributes switched on)	Final members after KNN
Heritage & Faith (Rome)	heritage, religious, art, nightlife	Barcelona, Istanbul, Rome, Vienna, Amsterdam, Prague
Art & Lifestyle (Paris)	heritage, art, shopping, nightlife	London, Milan, Paris
Street Food & Markets (Bangkok)	heritage, religious, street food, shopping, nightlife, skyline	Seoul, Tokyo, Bangkok, Osaka
Modern & Shopping (Dubai)	street food, shopping, nightlife, nature, skyline	Dubai, New York, Taipei, Singapore, Kuala Lumpur, Hong Kong, Shanghai

KNN moved 3 borderline cities: **Barcelona** (Art → Heritage), **Kuala Lumpur** and **Shanghai** (Street Food → Modern). It also labelled two cities that were never in the data: **Kyoto** (nearest: Istanbul, Rome, Vienna) and **Lisbon** (nearest: Amsterdam, Barcelona, Vienna). Both went to Heritage & Faith. K-means gives the shape, the seeds give the meaning, and KNN gives the fine-tuning and the reach to new data.

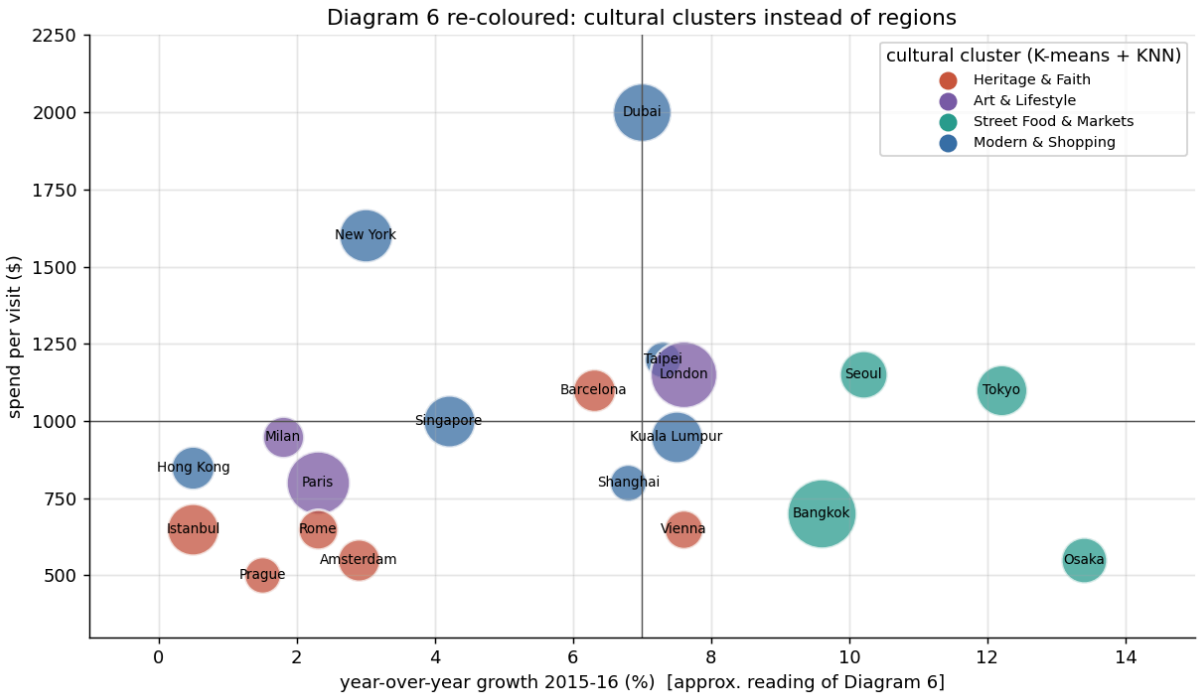


Figure 2. Diagram 6 redrawn from approximate readings, coloured by cultural cluster instead of region.

8. Organising a tour from a cultural perspective

For the culture lover, Heritage & Faith has the best average match. The real run gives:

- Match scores: Istanbul 0.88, Rome 0.88, Vienna 0.75, Prague 0.75, Barcelona 0.75, Amsterdam 0.75, Milan 0.62.
- The cross-cluster stop is **Milan** (Art & Lifestyle), the best in-budget match outside the main cluster.
- The tour has **7 stops and covers 4,752 km**: Istanbul → Vienna → Prague → Milan → Rome → Barcelona → Amsterdam.

Nearest-neighbour routing is greedy. The jump from Rome to Barcelona and then up to Amsterdam shows it is quick, not perfect. A 2-opt swap or the traveller's own edits can polish the order.

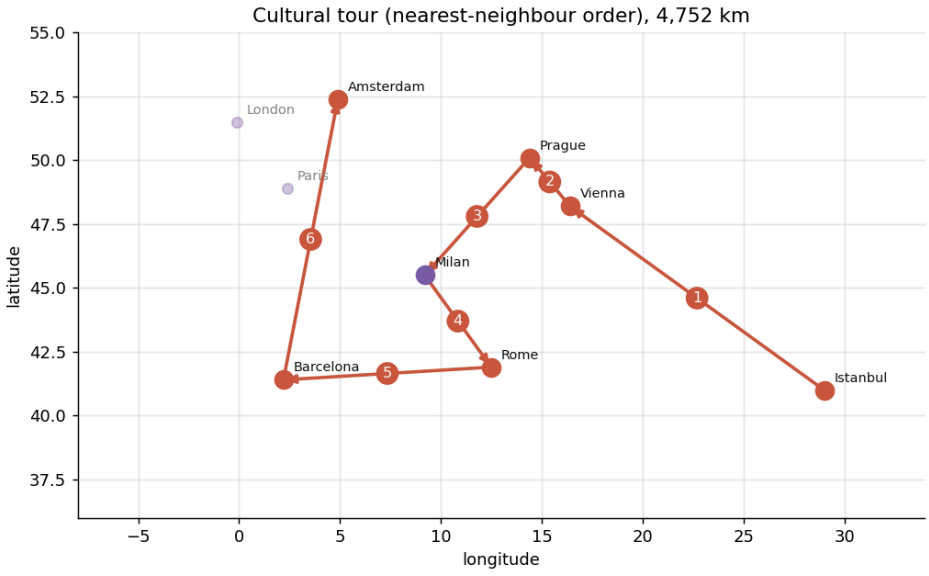


Figure 3. The planned route. Faded points are cities outside the tour.

9. Introducing systemic creativity

Systemic creativity means looking at the whole tourism system, not one city at a time. The links between parts are where new tour ideas come from:

- **Bridge cities.** The code measures how close each city is to its two nearest cluster centres. Ratios near 1 mean "sits between two cultures": **Barcelona (0.97), Osaka (0.82), Amsterdam (0.70)**. These are natural hinges for cross-cluster tours. Istanbul plays the same role geographically, joining Europe and Asia.
- **Deliberate cross-cluster stops.** Adding Milan to a heritage tour mixes art into faith. A small "negative" can lift the whole (Positive + Positive < Negative + Positive).
- **IoT and live data.** Footfall sensors, queue cameras and weather feeds can re-order stops to avoid crowds. Festival calendars add temporary attributes, so the clusters can be re-run each season.
- **Other domains.** Food, rail links, local events and hotel prices become extra dimensions. In practice, adding a data source expands the conceptual space of possible tours.

10. Diagram 6 modified to a cultural-preference distribution

The original chart places cities by *growth vs spend* and colours them by *continent*. Our version keeps spend on the y-axis and bubble size = visits. The x-axis becomes **cultural match with the traveller**, and the colour becomes the **cultural cluster**. The dashed lines (match 0.66, budget $1,200) mark a "sweet spot". The tour cities (black rings) sit at the bottom right, and Milan sits just outside as the planned cross-cluster stop.

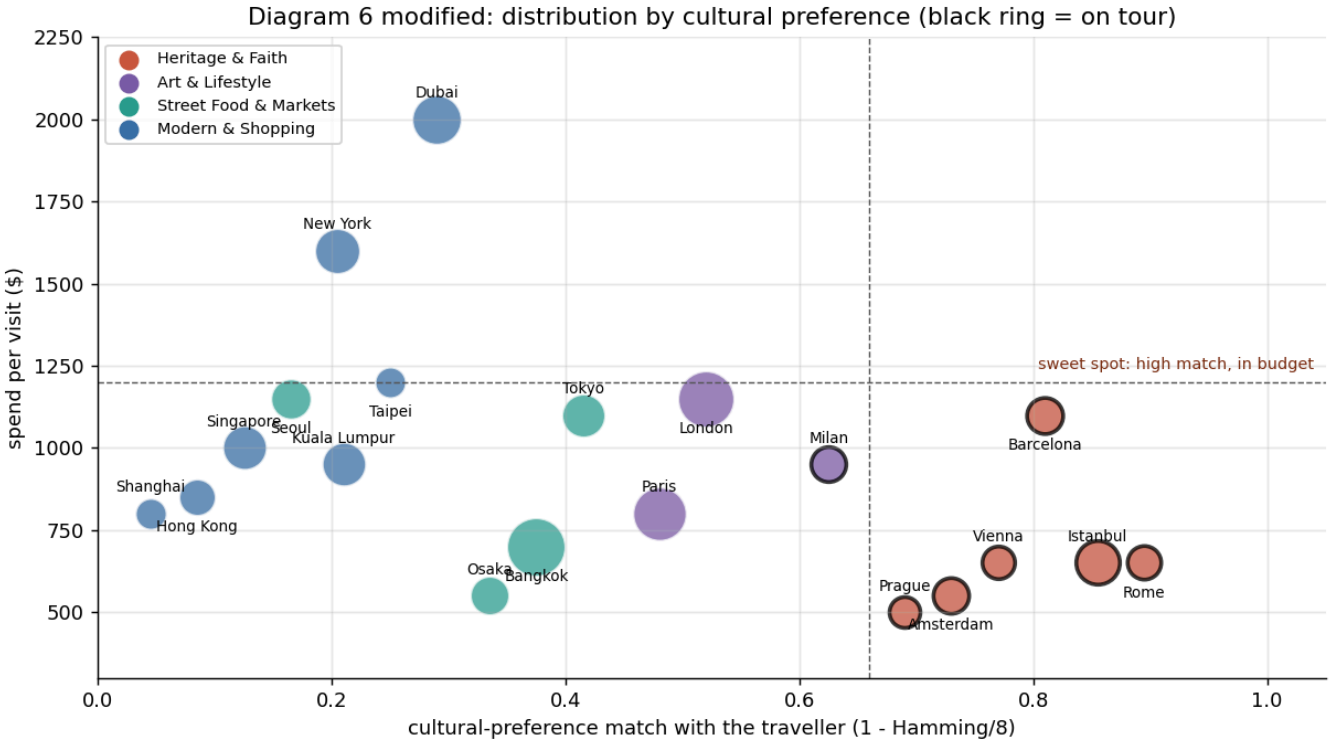


Figure 4. Diagram 6 modified: cities placed by cultural preference instead of growth.

11. Code and real run

`q14_code.py` does the mixed-distance K-means, seed naming, KNN refinement, new-city labelling, bridge detection and tour planning, and draws every chart above.

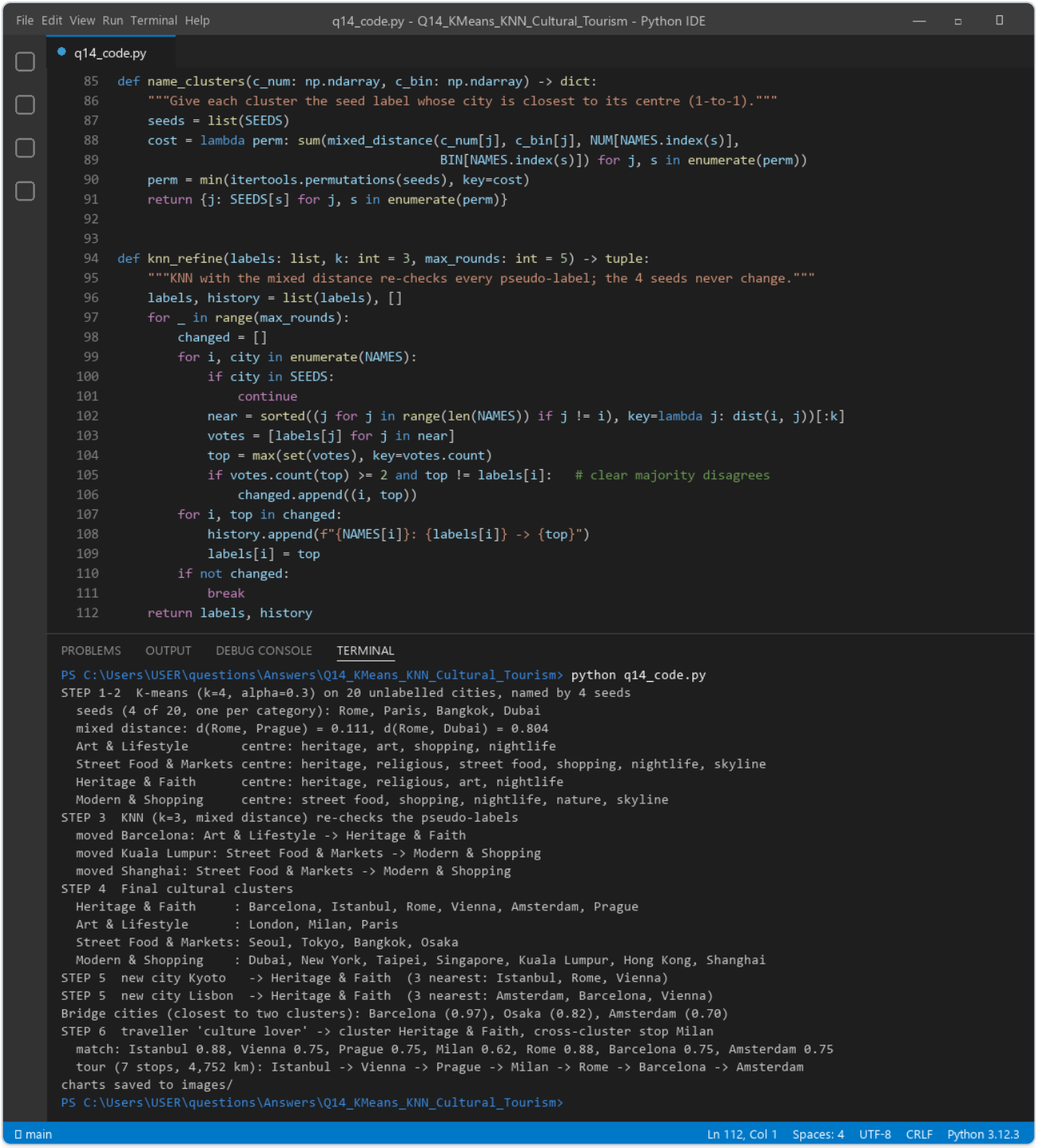


Figure 5. Cluster naming and the KNN refinement loop (lines 85-112), with the real terminal output.

12. Summary

> Yes, K-means and KNN combine naturally: K-means finds the cultural groups, 4 seeds name them, and KNN repairs borderline labels and classifies new places. A blended Euclidean + Hamming distance lets numbers and cultural yes/no facts work together. The result drives a cultural tour, and bridge cities plus live IoT data add the systemic, creative layer.

Files in this folder: `q14_code.py` (source), `q14_code.ipynb` (Colab/Jupyter version, already run), `q14_output.txt` (console output), `images/` (all figures).